GIT QUEST | 30-DAY GIT LEARNING SERIES

Day 11: What is a Branch?

Week 2: Undo and Branch | Level: Absolute Beginner

Created by Rutuja Naik

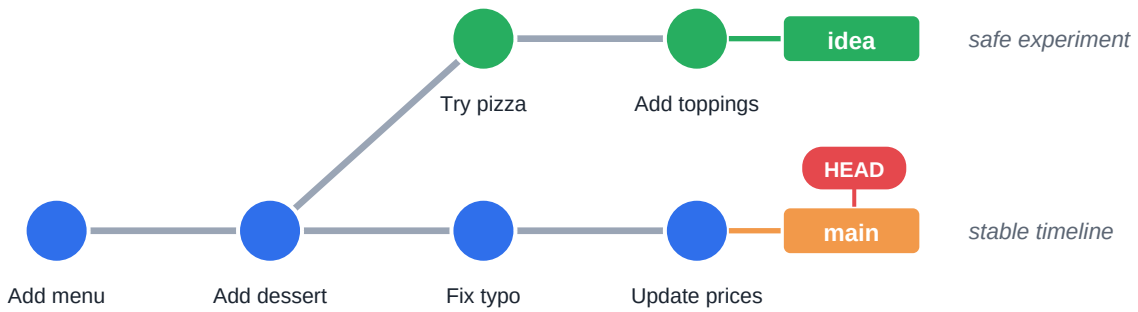
Today's mission

Today you meet one of the biggest ideas in Git: the **branch**. After everything you already know, it is simpler than it sounds. This is a concept day: tomorrow you will practise switching between branches.

- Explain what a branch is in plain words
- Understand `main` and `HEAD`
- Know why branches are useful
- List, create and delete a branch label

1. The big idea: a parallel timeline

So far you have had **one timeline**. What if you want to try a risky idea without spoiling the version that works? A **branch** is a separate line of work: a parallel timeline, or a spare save slot for experiments. If it goes wrong, your main timeline is untouched. If it goes well, you bring it back with a merge (Day 13).



The secret: a sticky note
A branch is **not a copy of your files**. It is only a tiny **movable label** that points at one commit. When you commit, the label moves forward to the new commit. That is why creating a branch is instant and almost free.

2. Key words

Word	What it really is
Branch	A sticky note stuck on a commit. It moves forward when you commit.
main	The default branch: the first sticky note, created by git init -b main.
HEAD	The "you are here" pin. It points at the branch you are on. HEAD -> main means "I am on main".

3. Why use branches?

Reason	Example
Try ideas safely	Test a new menu design without touching the working one
Work in parallel	One branch for the dessert page, another for prices
Keep main stable	main always contains work you trust
Collaborate	Each person works on their own branch (Week 3)

4. Step by step: meet your first branch

Step 1: Make a tiny history

```
$ git init -b main branch-lab
$ cd branch-lab
$ echo "Pasta" > menu.txt
$ git add menu.txt
$ git commit -m "Add menu"
$ echo "Dessert" >> menu.txt
$ git add menu.txt
$ git commit -m "Add dessert"
```

Step 2: See your only branch

```
$ git branch
* main
```

Step 3: Read the labels on your commits

```
$ git log --oneline --decorate
c3d9e41 (HEAD -> main) Add dessert
8a1f2b6 Add menu
```

Step 4: Stick a second label on the same commit

```
$ git branch idea
$ git log --oneline --decorate
c3d9e41 (HEAD -> main, idea) Add dessert
8a1f2b6 Add menu
```

Step 5: List branches again

```
$ git branch
  idea
* main
```

Step 6: Tidy up

```
$ git branch -d idea
Deleted branch idea (was c3d9e41).
```

Notice that `HEAD -> main` did not change: creating a branch does **not** move you onto it. Tomorrow you will learn `git switch` to jump between branches and commit on them.

5. Cheat sheet

Command	What it does
git branch	List branches (* marks the current one)
git branch <name>	Make a new branch label, you stay put
git branch -d <name>	Delete a branch label
git log --oneline --decorate	Show branch labels on commits
git log --oneline --graph --all	Draw every branch

> **Your Day 11 commit**
> File: `days-1-to-11-summary.md`
> Message: `docs(day11): add days 1-11 summary and branch notes`

6. Mermaid diagram GitHub draws for you

````
```mermaid
gitGraph
  commit id: "Add menu"
  commit id: "Add dessert"
  branch idea
  checkout idea
  commit id: "Try pizza"
  checkout main
  commit id: "Fix typo"
```
````

7. Challenge and quiz

Challenge: draw (on paper) a history with a "dark-mode" branch that has two commits while main gets one. Label HEAD.

Q1. What is a branch, really?

Q2. What does HEAD -> main mean?

Q3. Give two reasons to use branches.

Answers: 1) A movable label on one commit, not a copy. 2) You are on the main branch. 3) Try ideas safely, work in parallel, keep main stable.

Coming up: Day 12
git switch and git branch. Jump between branches and commit on them.